CampusConnect – College Event & Communication App

PROJECT ABSTRACT:

CampusConnect is a comprehensive iOS mobile application developed using UIKit and Storyboard that centralizes college event management, club communication, and student engagement within a single platform. The application enables students to discover events, create and manage clubs, register for activities, and maintain a personalized notification system while leveraging Core Data for local persistence and Firebase Authentication for secure user credential management. By providing an intuitive interface with real-time event filtering, club categorization, and event participation tracking, CampusConnect streamlines campus communication and fosters meaningful student connections across various academic and extracurricular activities.

PROBLEM STATEMENT:

College campuses face significant challenges in disseminating event information and coordinating club activities across diverse student groups. Traditional methods of communication—including email, notice boards, and fragmented social media channels—result in poor information accessibility, missed event participation opportunities, and reduced student engagement. Additionally, students lack a centralized platform to discover clubs matching their interests, connect with peers through organized events, and track their involvement in campus activities. The absence of a unified communication system creates information silos, making it difficult for students to stay informed about upcoming events or club initiatives while club organizers struggle to reach their target audience effectively.

PROPOSED SOLUTION:

CampusConnect addresses these challenges by providing a centralized, user-friendly mobile application that integrates event discovery, club management, and student participation tracking in one seamless platform. The application enables registered students to browse events categorized by club affiliation, search for specific activities using intelligent filtering mechanisms, and register for events with a single tap. Club administrators can create clubs, post events with detailed information including venue

location, date, time, and pricing, and receive notifications confirming student participation. The integration of Core Data ensures offline-first functionality and persistent storage of user preferences, event history, and club memberships, while Firebase Authentication provides secure account creation and sign-in workflows. Real-time notification systems keep users informed about new events, successful registrations, and club-related announcements, fostering a vibrant and connected campus community.

TECHNOLOGIES USED:

Category	**Technology**	**Purpose**
UI Framework	UIKit	Native iOS user interface components
Layout System	Storyboard & AutoLayout	Visual interface design and responsive constraints
Architecture Pattern	Model-View-Controller (MVC)	Separation of concerns and code organization
Local Database	CoreData	Persistent storage of events, clubs, users, and notifications
Authentication	Firebase Authentication	Secure sign-up, sign-in, and password reset functionality
UI Components	UITableView, UICollectionView	Data display in list and grid layouts
Navigation	UITabBarController, UINavigationController	Screen navigation and hierarchy management
Location Services	MapKit	Coordinate management for campus event locations
Image Handling	UIImagePickerController, FileManager	Event and profile image selection and storage

PROJECT ARCHITECTURE EXPLANATION:

Model-View-Controller (MVC) Architecture

CampusConnect follows the Model-View-Controller architectural pattern, which clearly separates application logic into three interconnected components:

Model Layer: The Model layer represents the data structures and business logic. Core Data entities define the data schema including EventEntity, ClubEntity, UserProfileEntity, ParticipationEntity, FavoriteEntity, and NotificationEntity. These entities store persistent information about events, clubs, user profiles, and user interactions. Data managers including CoreDataManager and AuthManager encapsulate all database operations and authentication logic, providing a single point of access for data operations across the application. This layer is completely independent of the user interface and focuses purely on data persistence and retrieval.

View Layer: The View layer consists of all user interface elements visible to the user. ViewControllers including HomeViewController, AddEventViewController, ClubFormationViewController, and EventDetailViewController manage the visual presentation of data. Custom cells such as EventCardCell, CategoryCell, MyEventCell, and MyClubCell format data for display in collections and tables. UI components like UITextField, UIButton, UILabel, UIImageView, UIPickerView, and UIDatePicker handle user interactions. All views are designed using Storyboard with AutoLayout constraints, ensuring responsive layouts across different device sizes. The View layer is purely presentational and contains no business logic.

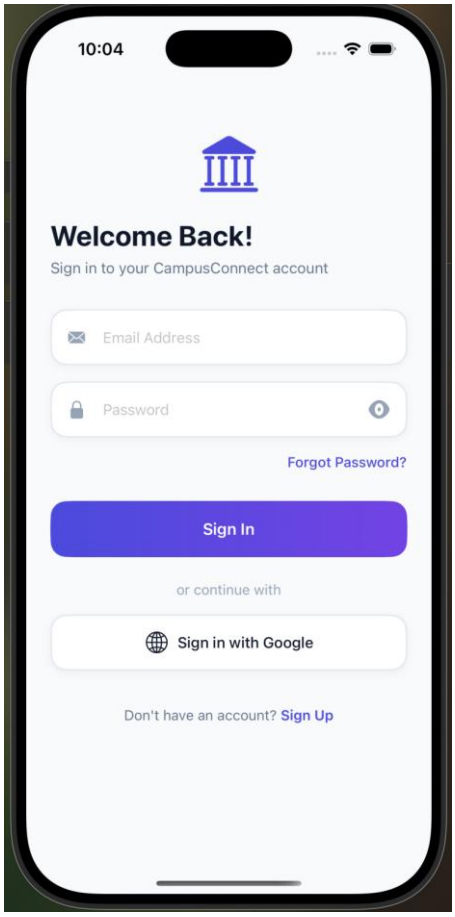
Controller Layer: The Controller layer acts as a bridge between Models and Views, handling user interactions and updating the UI based on model changes. ViewControllers manage user input from text fields, button taps, and picker selections. They query the Model layer through managers (CoreDataManager, AuthManager, SessionManager) to fetch or modify data, then update Views to reflect these changes. Controllers also coordinate navigation between screens using UINavigationController and UIStoryboardSegue. Event handlers such as @objc methods respond to user actions like button taps and gesture recognitions, triggering appropriate data operations and view updates.

Data Flow:

User Input → Controller → Manager → Core Data/Firebase → Manager → Controller → View Update

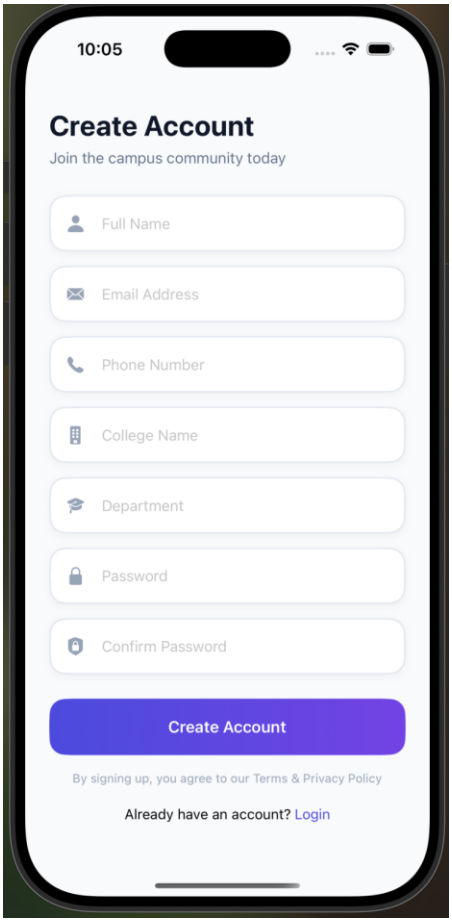
This layered approach ensures loose coupling between components, making the code maintainable, testable, and scalable.

SignIn View Controller



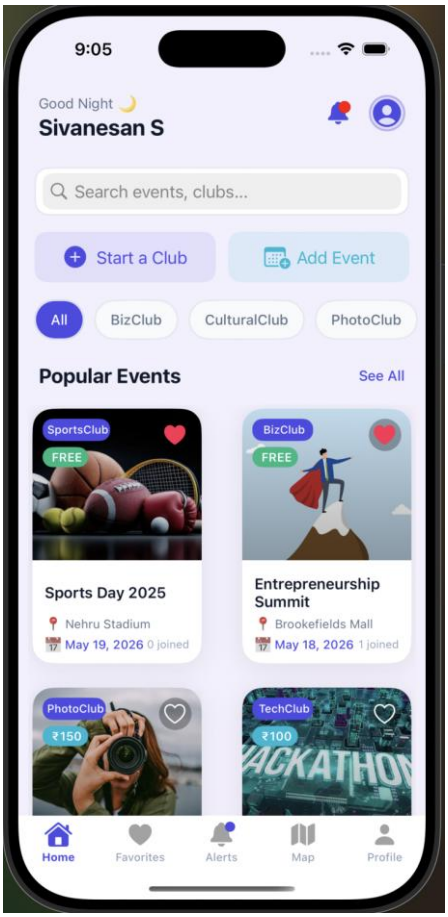
- **Sign In Screen** authenticates existing users using email and password through Firebase Authentication while managing secure session initialization.
- The interface includes validated email/password fields, password visibility toggle, forgot password support, and a gradient-based sign-in button with loading indicator feedback.
- Successful login stores user session details such as UID, email, and display name using SessionManager before navigating to the MainTabBar with a smooth transition animation.
- The screen follows a clean form-based UIKit design using rounded container views, icon-enhanced text fields, and structured navigation for an intuitive authentication experience.

SignUp View Controller



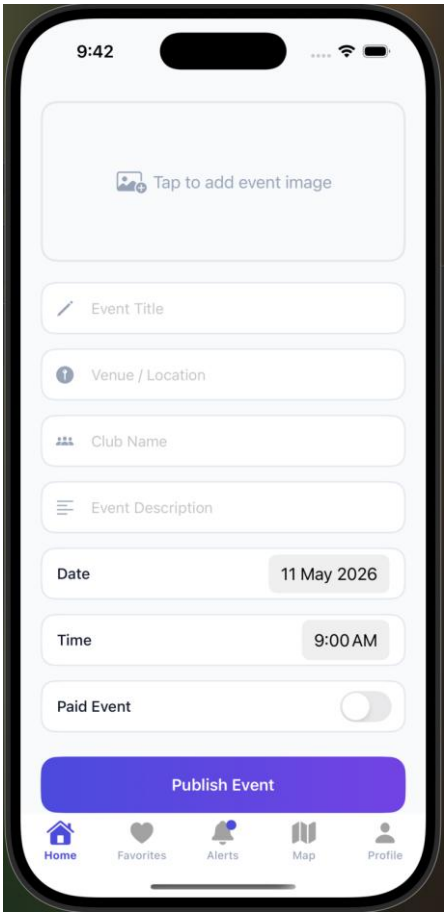
- Sign Up Screen enables new users to create accounts by collecting details such as name, email, phone number, college, department, and password through a structured multi-field form.
- The controller performs real-time validation including empty field checks, email verification, password strength validation, and password confirmation matching before Firebase registration.
- After successful signup, user profile information is stored using Core Data and SessionManager, followed by OTP-based email verification through a modal verification screen.
- The interface follows a clean UIKit form design with icon-based input fields, rounded container views, loading indicators, and smooth navigation between authentication screens and the MainTabBar.

Home View Controller



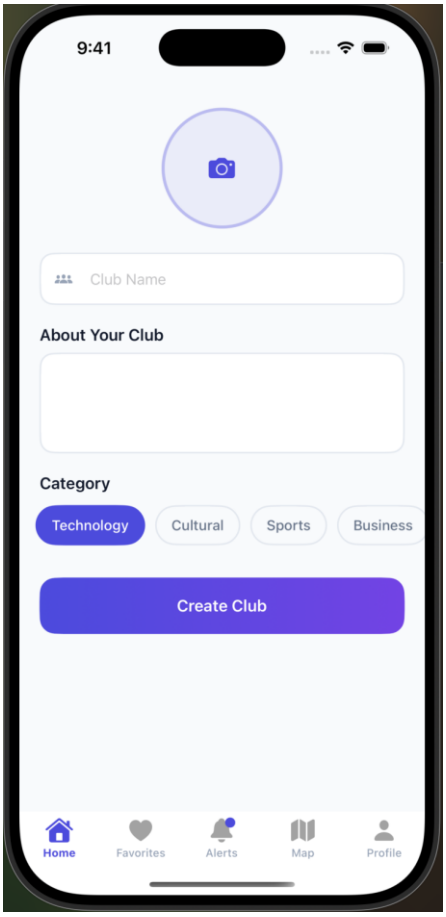
- Home Screen acts as the main event discovery hub with personalized greetings, category-based event filtering, and real-time search functionality for events and clubs.
- The controller displays events in a dynamic collection view grid with event details, favorite toggling, notification badges, and quick actions for creating clubs or events.
- Data is managed using Core Data for events, favorites, clubs, and notifications, while SessionManager provides personalized user information and profile display.
- The interface follows a modern UIKit design using collection views, gradient backgrounds, rounded cards, category pills, and smooth tab-based navigation for an engaging user experience.

AddEvent View Controller



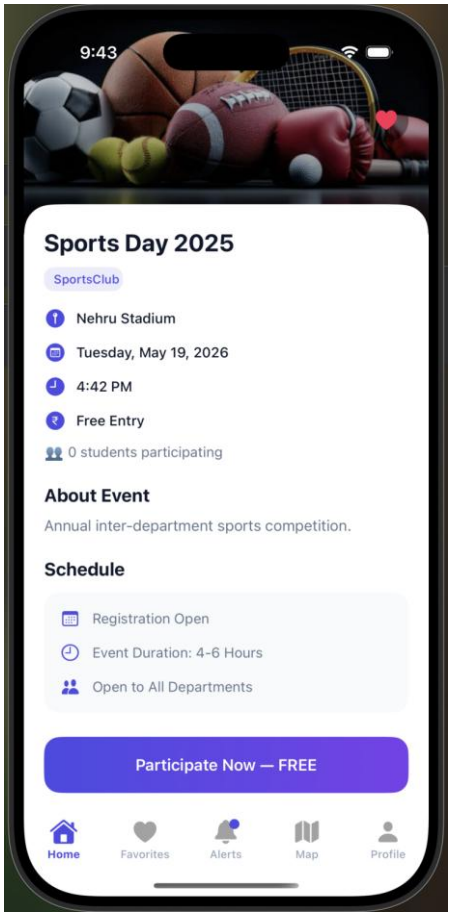
- Add Event Screen allows organizers to create and publish campus events by entering details such as title, venue, club, description, date/time, pricing, and event images.
- The controller uses picker views for predefined locations and clubs, image selection through UIImagePickerController, and conditional pricing fields for paid events.
- Event information, images, coordinates, and notifications are stored using Core Data and FileManager, ensuring persistent event management and retrieval.
- The interface follows a clean UIKit form-based design with rounded input containers, icon-enhanced fields, gradient action buttons, and scrollable layouts for smooth user interaction.

ClubFormation View Controller



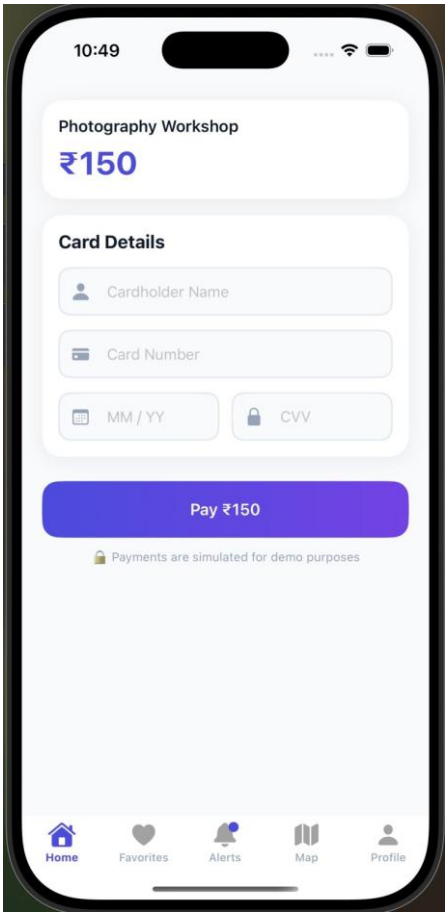
- Create Club Screen enables students to establish new campus clubs by entering club details such as name, description, category, and custom club image.
- The controller uses a horizontal category collection view for selecting predefined club categories and UIImagePickerController for club logo or banner selection.
- Club information and selected images are stored using Core Data and FileManager, while notifications are generated to confirm successful club creation.
- The interface follows a clean UIKit design with scrollable layouts, circular image selection, category pill cells, rounded input fields, and gradient action buttons for intuitive interaction.

EventDetail View Controller



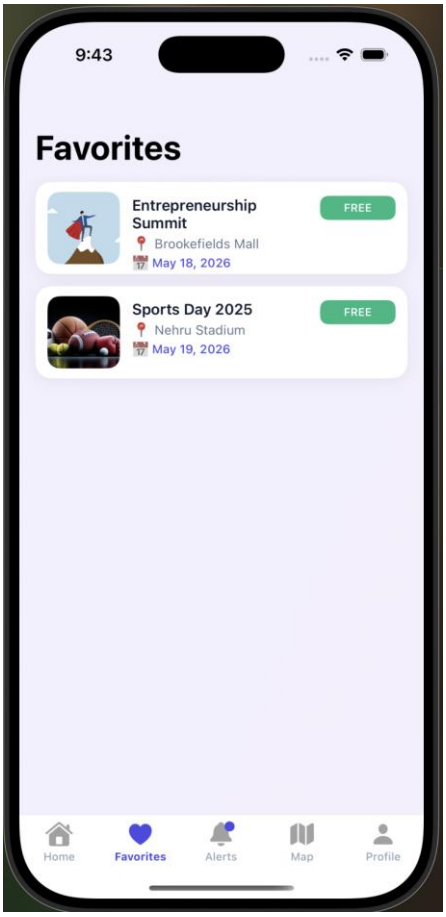
- Event Details Screen displays complete event information including banner image, event description, venue, date/time, pricing, participant count, and club details in a scrollable card-based layout.
- The controller supports favorite management, participation tracking, and registration workflows with separate handling for free and paid events through PaymentViewController integration.
- Event participation, favorites, and notifications are managed using Core Data, while event images are loaded dynamically from FileManager storage.
- The interface follows a modern UIKit design with gradient overlays, rounded information cards, icon-based detail sections, animated favorite actions, and dynamic participation button states for enhanced user interaction.

Payment View Controller



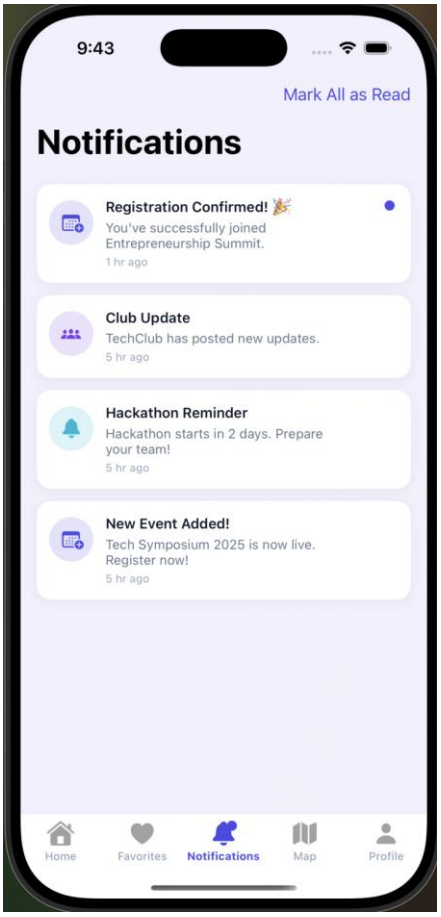
- Payment and Success Screens manage the simulated transaction workflow for paid events, including payment validation, registration handling, and confirmation feedback.
- The payment controller validates card details using sequential input checks and the Luhn algorithm, while supporting expiry selection through a picker-based interface.
- Successful payments store participation records and notifications using Core Data, prevent duplicate registrations, and navigate users to a dedicated success confirmation screen.
- The interface follows a modern UIKit card-based design with gradient action buttons, secure payment-style forms, animated success indicators, and smooth navigation transitions back to the Home screen.

Favorites View Controller



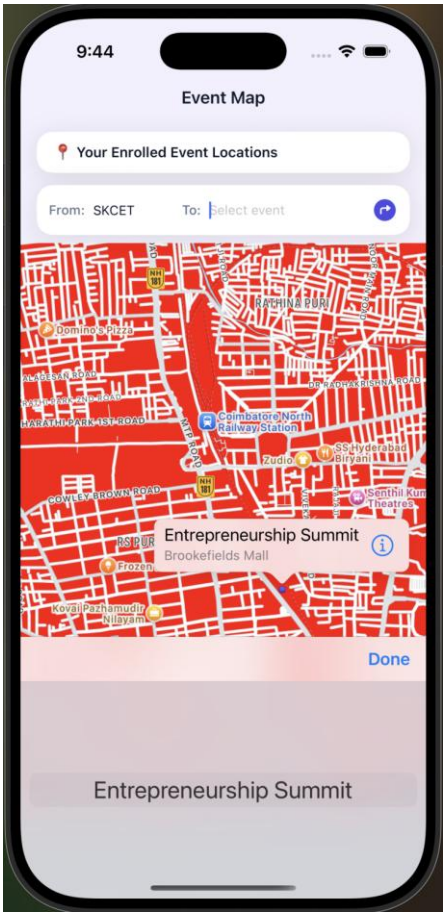
- Favorites Screen displays the user's saved events in a clean table-based layout, allowing quick access to bookmarked campus events and activities.
- The controller dynamically fetches favorite events from Core Data, supports swipe-to-delete functionality, and updates the UI instantly after removal actions.
- Selecting an event navigates users to the detailed event information screen, while locally stored images are loaded using FileManager utilities.
- The interface follows a modern UIKit design with custom table cells, subtle gradient backgrounds, and a dedicated empty-state view for improved user experience.

Notification View Controller



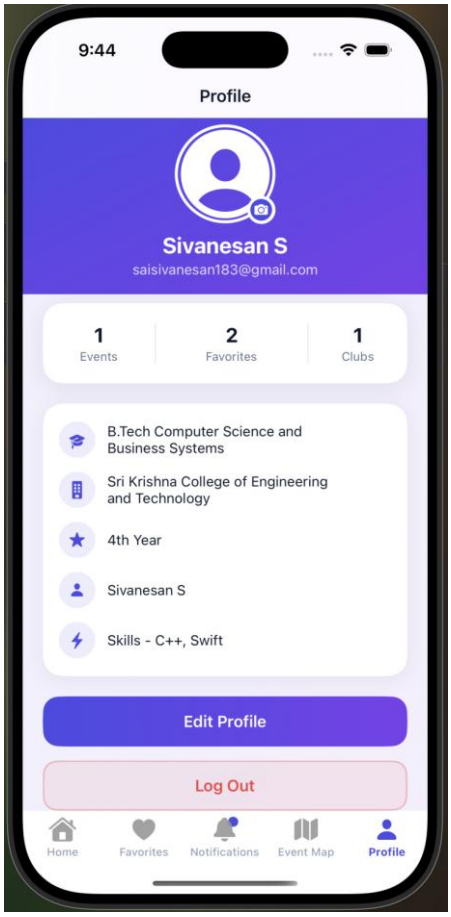
- Notifications Screen displays a chronological list of event and club-related notifications with support for read and unread state management.
- The controller retrieves notification data from Core Data and updates notification badges dynamically through NotificationCenter observers integrated with the Home screen.
- Users can interact with notifications through selection or swipe actions, enabling navigation to related event details or removal of notifications.
- The interface is designed using a clean UIKit table-based layout with timestamped notification cells, structured organization, and seamless tab-based navigation integration.

Map View Controller



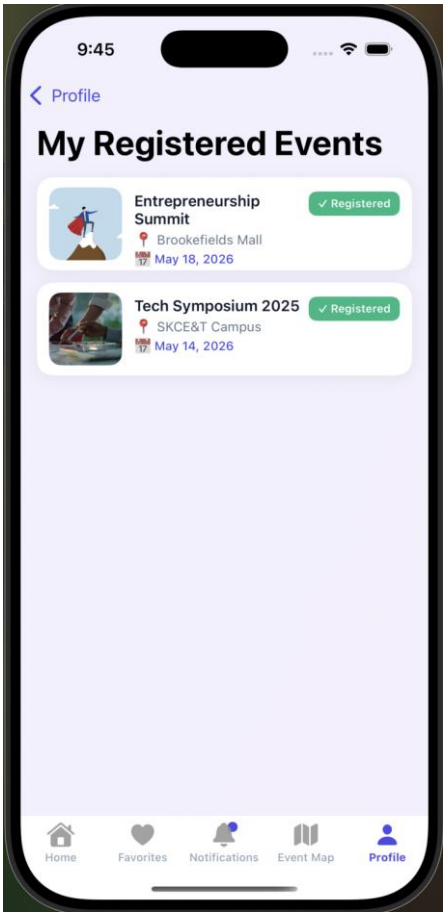
- Event Map Screen provides a geographical overview of enrolled events using Apple Maps with interactive event markers and built-in route navigation functionality.
- The controller integrates MapKit to display event locations, calculate routes from the college campus to selected venues, and render navigation paths using MKDirections overlays.
- Registered event data is retrieved from Core Data, while custom map annotations pass EventEntity information for seamless navigation to detailed event screens.
- The interface follows a modern floating-card UIKit design with gradient overlays, interactive map controls, custom marker styling, and integrated routing tools for enhanced user experience.

Profile View Controller



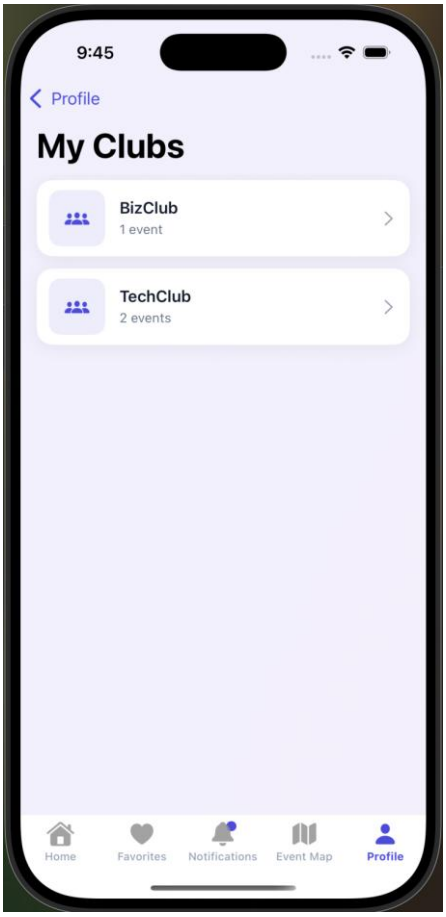
- User Profile Screen serves as the central hub for displaying and managing user information, profile customization, participation statistics, and account-related actions.
- The controller supports profile image updates, editable user details, dynamic statistics for events, favorites, and clubs, along with secure logout functionality.
- User data, profile images, and activity statistics are managed using Core Data, SessionManager, and FileManager for persistent storage and retrieval.
- The interface follows a modern UIKit card-based design with gradient headers, overlapping statistic cards, rounded profile images, and smooth navigation to related profile sections.

MyEvents View Controller



- My Events Screen displays all events registered by the current user, providing quick access to participation history and event details.
- The controller retrieves participation records from Core Data, filters registered events dynamically, and updates the table view whenever the screen reappears.
- Users can select any registered event to navigate to the detailed event information screen, while empty-state handling guides users when no registrations exist.
- The interface follows a clean UIKit table-based design with custom event cells, gradient backgrounds, large navigation titles, and structured event previews for improved usability.

MyClubs View Controller



- My Clubs Screen displays all clubs associated with the current user, along with the number of events available within each club.
- The controller retrieves joined club information and related event counts dynamically from Core Data, updating the table whenever the screen appears.
- Users can select a club to navigate to a dedicated screen showing all events related to the selected club, while empty-state handling guides inactive users.
- The interface follows a clean UIKit table-based design with custom club cells, gradient backgrounds, large navigation titles, and structured event count previews for better usability.

CONCLUSION:

CampusConnect is a professionally designed iOS application that integrates UIKit, Core Data, Firebase Authentication, and MapKit to create a centralized platform for campus event and club management. The project follows strong software engineering practices using MVC architecture, reusable UI components, singleton-based managers, and responsive UIKit design patterns for maintainable and scalable development.

The application delivers key features including event discovery, club formation, participation tracking, favorites management, notifications, payment workflows, and personalized user profiles, creating a complete campus engagement ecosystem. Advanced integrations such as image handling, dynamic filtering, location mapping, validation mechanisms, and smooth navigation demonstrate practical iOS development expertise and attention to user experience.

The project also establishes a scalable foundation for future enhancements such as real-time messaging, push notifications, backend APIs, and calendar synchronization. Overall, CampusConnect demonstrates intermediate-to-advanced iOS development competency suitable for professional portfolios, internships, and real-world application development scenarios.